RandomForest_Estres

May 10, 2026

1 Random Forest Regressor — Estrés en Adolescentes

Variable dependiente cuantitativa: `stress_level` (nivel de estrés, escala 1-10)
Dataset: Teen Mental Health Dataset (1,200 observaciones)
Objetivo: Predecir el nivel de estrés a partir de variables de uso de redes sociales, sueño, rendimiento académico y salud mental.

Antes de ejecutar: Sube el archivo `datosestresyansiedad.xlsx` a Google Colab:
- Menú lateral → Archivos → Subir - O ejecuta la celda de carga con `files.upload()`

[3]:
```
# ============================================================================
#  SECCIÓN 0 - LIBRERÍAS
# ============================================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import warnings

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     cross_val_score, learning_curve)
from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             r2_score, mean_absolute_percentage_error)
from sklearn.inspection import permutation_importance
from sklearn.preprocessing import LabelEncoder
from scipy import stats

warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-whitegrid')
SEED = 42
np.random.seed(SEED)
print('  Librerías cargadas')
```

```
 Librerías cargadas
```

[4]:
```
# ============================================================================
#  CARGA DEL ARCHIVO - elige una de las dos opciones
# ============================================================================

# --- OPCIÓN A: subida manual desde el panel de archivos de Colab ---
# (ya subiste el .xlsx al panel de archivos → usa esta línea)
df = pd.read_excel('datosestresyansiedad.xlsx')

# --- OPCIÓN B: subida interactiva (descomenta si prefieres el diálogo) ---
# from google.colab import files
# uploaded = files.upload()          # selecciona datosestresyansiedad.xlsx
# df = pd.read_excel(list(uploaded.keys())[0])

print(f' Dataset cargado: {df.shape[0]} filas × {df.shape[1]} columnas')
```

```
Dataset cargado: 1200 filas × 13 columnas
```

1.1 Sección 1 — Auditoría del Dataset

[5]:
```
# ============================================================================
#  SECCIÓN 1 - CARGA Y AUDITORÍA DEL DATASET
# ============================================================================

print('\n' + '='*60)
print('  AUDITORÍA DEL DATASET')
print('='*60)
print(f'  Filas        : {df.shape[0]}')
print(f'  Columnas     : {df.shape[1]}')
print(f'  Duplicados   : {df.duplicated().sum()}')
print(f'  Valores nulos: {df.isnull().sum().sum()}')
print(f'\n  Tipos de datos:\n{df.dtypes.to_string()}')

# Diccionario de variables
descripcion = {
    'age'                    : 'Edad del adolescente (13-19 años)',
    'gender'                 : 'Género (male / female)',
    'daily_social_media_hours' : 'Horas diarias en redes sociales',
    'platform_usage'         : 'Plataforma principal (Instagram / TikTok /␣
 ↪Both)',
    'sleep_hours'            : 'Horas de sueño por noche',
    'screen_time_before_sleep' : 'Tiempo de pantalla antes de dormir (horas)',
    'academic_performance'   : 'Rendimiento académico (GPA aproximado)',
    'physical_activity'      : 'Horas de actividad física por día',
    'social_interaction_level' : 'Nivel de interacción social (low / medium /␣
 ↪high)',
    'anxiety_level'          : 'Nivel de ansiedad (escala 1-10)',
    'addiction_level'        : 'Nivel de adicción a redes (escala 1-10)',
```

```
    'depression_label'         : 'Etiqueta de depresión (0=no / 1=sí)',
    'stress_level'             : '  NIVEL DE ESTRÉS (escala 1-10) - TARGET'
}
print('\n  Diccionario de variables:')
for col, desc in descripcion.items():
    print(f'    {col:<30}: {desc}')

TARGET   = 'stress_level'
FEATURES_RAW = [c for c in df.columns if c != TARGET]
```

```
============================================================
  AUDITORÍA DEL DATASET
============================================================
  Filas        : 1200
  Columnas     : 13
  Duplicados   : 0
  Valores nulos: 0

  Tipos de datos:
age                           int64
gender                       object
daily_social_media_hours    float64
platform_usage               object
sleep_hours                 float64
screen_time_before_sleep    float64
academic_performance        float64
physical_activity           float64
social_interaction_level     object
anxiety_level                 int64
addiction_level               int64
depression_label              int64
stress_level                  int64

  Diccionario de variables:
    age                          : Edad del adolescente (13-19 años)
    gender                       : Género (male / female)
    daily_social_media_hours     : Horas diarias en redes sociales
    platform_usage               : Plataforma principal (Instagram / TikTok /
Both)
    sleep_hours                  : Horas de sueño por noche
    screen_time_before_sleep     : Tiempo de pantalla antes de dormir (horas)
    academic_performance         : Rendimiento académico (GPA aproximado)
    physical_activity            : Horas de actividad física por día
    social_interaction_level     : Nivel de interacción social (low / medium /
high)
    anxiety_level                : Nivel de ansiedad (escala 1-10)
```

```
    addiction_level                 : Nivel de adicción a redes (escala 1-10)
    depression_label                : Etiqueta de depresión (0=no / 1=sí)
    stress_level                    :   NIVEL DE ESTRÉS (escala 1-10) - TARGET
```

1.2 Sección 2 — Análisis Exploratorio (EDA)

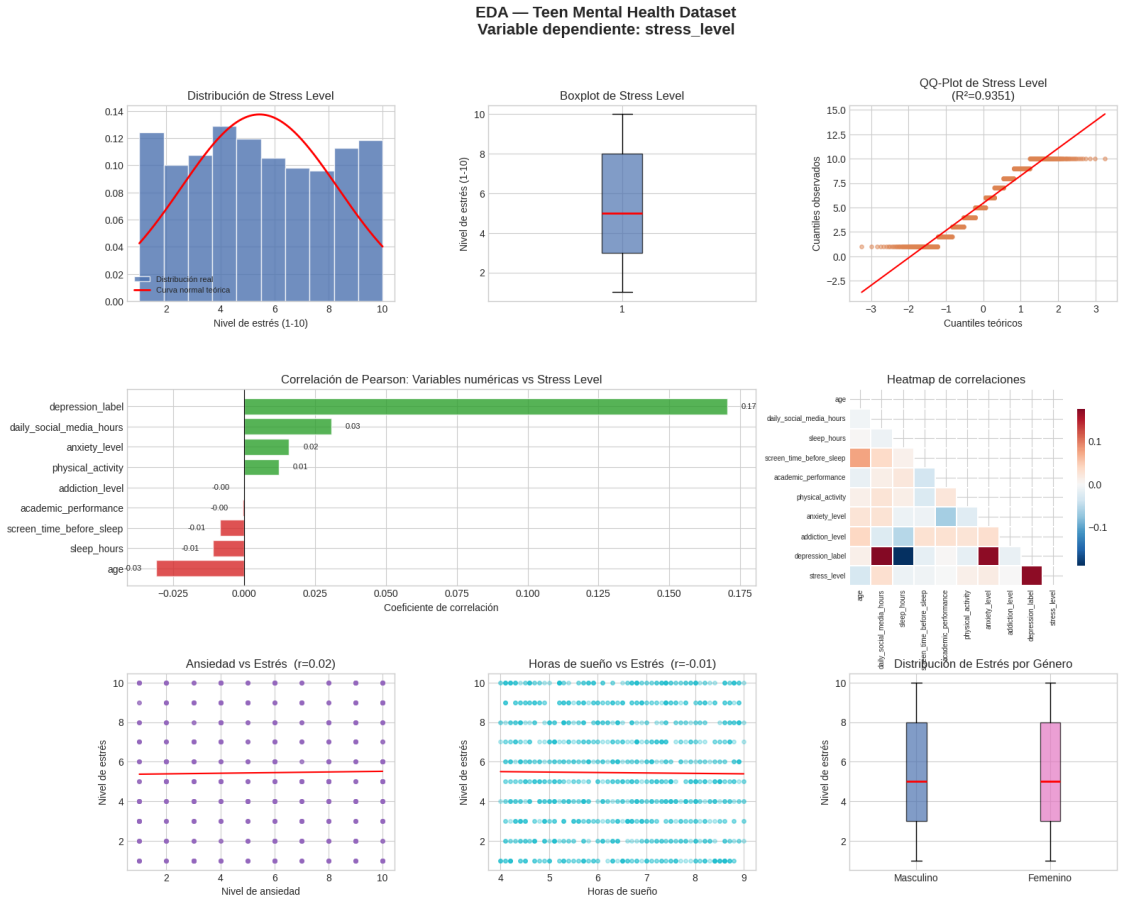
[6]:
```
# ============================================================================
#  SECCIÓN 2 - ANÁLISIS EXPLORATORIO (EDA)
# ============================================================================

# 2.1 Estadísticas del target
print('\n' + '='*55)
print('  ANÁLISIS DE LA VARIABLE DEPENDIENTE: stress_level')
print('='*55)
print(f'  Media    : {df[TARGET].mean():.2f}')
print(f'  Mediana  : {df[TARGET].median():.2f}')
print(f'  Std      : {df[TARGET].std():.2f}')
print(f'  Min      : {df[TARGET].min():.0f}')
print(f'  Max      : {df[TARGET].max():.0f}')
print(f'  Skewness : {df[TARGET].skew():.4f}  (>0 = cola derecha)')
print(f'  Kurtosis : {df[TARGET].kurt():.4f}')
print(f'\n  Distribución por nivel:')
print(df[TARGET].value_counts().sort_index().to_string())
```

```
=======================================================
  ANÁLISIS DE LA VARIABLE DEPENDIENTE: stress_level
=======================================================
  Media    : 5.45
  Mediana  : 5.00
  Std      : 2.90
  Min      : 1
  Max      : 10
  Skewness : 0.0476  (>0 = cola derecha)
  Kurtosis : -1.2152

  Distribución por nivel:
stress_level
1     134
2     108
3     116
4     139
5     129
6     114
7     106
8     104
9     122
10    128
```

[7]:
```
# 2.2 Panel EDA completo
fig = plt.figure(figsize=(18, 14))
gs  = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)
fig.suptitle('EDA - Teen Mental Health Dataset\nVariable dependiente:␣
 ↪stress_level',
             fontsize=16, fontweight='bold', y=0.98)

# (a) Distribución del target
ax1 = fig.add_subplot(gs[0, 0])
ax1.hist(df[TARGET], bins=10, color='#4C72B0', edgecolor='white',
         alpha=0.8, density=True, label='Distribución real')
mu, sigma = df[TARGET].mean(), df[TARGET].std()
x_norm = np.linspace(df[TARGET].min(), df[TARGET].max(), 200)
ax1.plot(x_norm, stats.norm.pdf(x_norm, mu, sigma),
         'r-', linewidth=2, label='Curva normal teórica')
ax1.set_title('Distribución de Stress Level')
ax1.set_xlabel('Nivel de estrés (1-10)')
ax1.legend(fontsize=8)

# (b) Boxplot del target
ax2 = fig.add_subplot(gs[0, 1])
ax2.boxplot(df[TARGET], vert=True, patch_artist=True,
            boxprops=dict(facecolor='#4C72B0', alpha=0.7),
            medianprops=dict(color='red', linewidth=2))
ax2.set_title('Boxplot de Stress Level')
ax2.set_ylabel('Nivel de estrés (1-10)')

# (c) QQ-Plot
ax3 = fig.add_subplot(gs[0, 2])
(osm, osr), (slope, intercept, r) = stats.probplot(df[TARGET], dist='norm')
ax3.scatter(osm, osr, alpha=0.5, s=15, color='#dd8452')
ax3.plot(osm, slope * np.array(osm) + intercept, 'r-', linewidth=1.5)
ax3.set_title(f'QQ-Plot de Stress Level\n(R²={r**2:.4f})')
ax3.set_xlabel('Cuantiles teóricos')
ax3.set_ylabel('Cuantiles observados')

# (d) Correlación numérica con target
ax4 = fig.add_subplot(gs[1, :2])
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
corr = df[numeric_cols].corr()[TARGET].drop(TARGET).sort_values()
colors_c = ['#d62728' if v < 0 else '#2ca02c' for v in corr]
bars = ax4.barh(corr.index, corr.values, color=colors_c, alpha=0.8,␣
 ↪edgecolor='white')
ax4.axvline(0, color='black', linewidth=0.8)
ax4.set_title('Correlación de Pearson: Variables numéricas vs Stress Level')
ax4.set_xlabel('Coeficiente de correlación')
for bar, val in zip(bars, corr.values):
```

```
    ax4.text(val + (0.005 if val >= 0 else -0.005), bar.get_y() + bar.
 ↪get_height()/2,
             f'{val:.2f}', va='center', ha='left' if val >= 0 else 'right',␣
 ↪fontsize=8)

# (e) Heatmap de correlaciones numéricas
ax5 = fig.add_subplot(gs[1, 2])
corr_matrix = df[numeric_cols].corr()
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, ax=ax5, cmap='RdBu_r',
            center=0, annot=False, linewidths=0.3, cbar_kws={'shrink': 0.8})
ax5.set_title('Heatmap de correlaciones')
ax5.tick_params(labelsize=7)

# (f) Scatter: anxiety_level vs stress_level
ax6 = fig.add_subplot(gs[2, 0])
ax6.scatter(df['anxiety_level'], df[TARGET], alpha=0.3, color='#9467bd', s=15)
m, b, r_val, *_ = stats.linregress(df['anxiety_level'], df[TARGET])
x_line = np.linspace(df['anxiety_level'].min(), df['anxiety_level'].max(), 100)
ax6.plot(x_line, m*x_line + b, 'r-', linewidth=1.5)
ax6.set_title(f'Ansiedad vs Estrés  (r={r_val:.2f})')
ax6.set_xlabel('Nivel de ansiedad')
ax6.set_ylabel('Nivel de estrés')

# (g) Scatter: sleep_hours vs stress_level
ax7 = fig.add_subplot(gs[2, 1])
ax7.scatter(df['sleep_hours'], df[TARGET], alpha=0.3, color='#17becf', s=15)
m2, b2, r_val2, *_ = stats.linregress(df['sleep_hours'], df[TARGET])
x_line2 = np.linspace(df['sleep_hours'].min(), df['sleep_hours'].max(), 100)
ax7.plot(x_line2, m2*x_line2 + b2, 'r-', linewidth=1.5)
ax7.set_title(f'Horas de sueño vs Estrés  (r={r_val2:.2f})')
ax7.set_xlabel('Horas de sueño')
ax7.set_ylabel('Nivel de estrés')

# (h) Estrés por género (boxplot)
ax8 = fig.add_subplot(gs[2, 2])
grupos = [df[df['gender'] == g][TARGET].values for g in ['male', 'female']]
ax8.boxplot(grupos, labels=['Masculino', 'Femenino'], patch_artist=True,
            boxprops=dict(alpha=0.7),
            medianprops=dict(color='red', linewidth=2))
for patch, color in zip(ax8.patches, ['#4C72B0', '#e377c2']):
    patch.set_facecolor(color)
ax8.set_title('Distribución de Estrés por Género')
ax8.set_ylabel('Nivel de estrés')

plt.savefig('01_eda_completo.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
print('  EDA guardado → 01_eda_completo.png')
```




```
EDA guardado → 01_eda_completo.png
```

1.3 Sección 3 — Preprocesamiento

[8]:
```
# ============================================================================
#  SECCIÓN 3 - PREPROCESAMIENTO
#  Codificación de variables categóricas + split 80/20
# ============================================================================

df_model = df.copy()

#   3.1 Codificación de variables categóricas con Label Encoding
cat_cols = ['gender', 'platform_usage', 'social_interaction_level']
encoders = {}

print('\n  CODIFICACIÓN DE VARIABLES CATEGÓRICAS')
print('  ' + '-'*45)
```

```python
for col in cat_cols:
    le = LabelEncoder()
    df_model[col] = le.fit_transform(df_model[col])
    encoders[col] = le
    print(f'  {col:<30}: {dict(zip(le.classes_, le.transform(le.classes_)))}')

#   3.2 Features y target
FEATURES = [c for c in df_model.columns if c != TARGET]
X = df_model[FEATURES].copy()
y = df_model[TARGET].copy()

#   3.3 Detección de outliers en features numéricas
print('\n' + '='*55)
print('  OUTLIERS POR FEATURE (z-score > 3)')
print('='*55)
for col in FEATURES:
    if X[col].nunique() > 10:  # solo para continuas
        n_out = (np.abs(stats.zscore(X[col])) > 3).sum()
        if n_out > 0:
            print(f'  {col:<30}: {n_out} outliers')

#   3.4 Split 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)
print(f'\n  Train: {X_train.shape[0]} muestras ({X_train.shape[0]/len(df)*100:.
  ↪0f}%)')
print(f'  Test : {X_test.shape[0]} muestras  ({X_test.shape[0]/len(df)*100:.
  ↪0f}%)')
print(f'\n  Media Stress - Train: {y_train.mean():.2f} | Test: {y_test.mean():.
  ↪2f}')
print(f'  Std  Stress - Train: {y_train.std():.2f}  | Test: {y_test.std():.2f}')
```

```
  CODIFICACIÓN DE VARIABLES CATEGÓRICAS
  ----------------------------------------------
  gender                         : {'female': np.int64(0), 'male': np.int64(1)}
  platform_usage                 : {'Both': np.int64(0), 'Instagram':
np.int64(1), 'TikTok': np.int64(2)}
  social_interaction_level       : {'high': np.int64(0), 'low': np.int64(1),
'medium': np.int64(2)}

=======================================================
  OUTLIERS POR FEATURE (z-score > 3)
=======================================================

  Train: 960 muestras (80%)
```

```
Test : 240 muestras  (20%)

Media Stress - Train: 5.42 | Test: 5.57
Std  Stress - Train: 2.93  | Test: 2.79
```

1.4 Sección 4 — Función de Evaluación

[9]:
```python
# ============================================================================
#  SECCIÓN 4 - FUNCIÓN DE EVALUACIÓN COMPLETA
# ============================================================================

def evaluar_modelo(nombre, y_true, y_pred, verbose=True):
    """
    Calcula métricas completas para regresión.
    - RMSE  : Error cuadrático medio (misma escala que stress_level)
    - MAE   : Error absoluto medio
    - MAPE  : Error porcentual absoluto medio
    - R²    : Proporción de varianza explicada
    - R²adj : R² ajustado por número de features
    """
    n, p    = len(y_true), X_train.shape[1]
    rmse    = np.sqrt(mean_squared_error(y_true, y_pred))
    mae     = mean_absolute_error(y_true, y_pred)
    mape    = mean_absolute_percentage_error(y_true, y_pred) * 100
    r2      = r2_score(y_true, y_pred)
    r2_adj = 1 - (1 - r2) * (n - 1) / (n - p - 1)

    if verbose:
        print(f"\n{'='*50}")
        print(f'    {nombre}')
        print(f"{'='*50}")
        print(f'  RMSE    : {rmse:.4f}  ← error en puntos de estrés')
        print(f'  MAE     : {mae:.4f}  ← error absoluto medio')
        print(f'  MAPE    : {mape:.2f}%  ← error porcentual')
        print(f'  R²      : {r2:.4f}  ← {r2*100:.1f}% varianza explicada')
        print(f'  R² adj  : {r2_adj:.4f}  ← penaliza por nº features')

    return {'RMSE': rmse, 'MAE': mae, 'MAPE': mape, 'R2': r2, 'R2_adj': r2_adj}
```

1.5 Sección 5 — Modelo Base

[10]:
```python
# ============================================================================
#  SECCIÓN 5 - MODELO BASE
# ============================================================================

print('\n modelo base')
rf_base = RandomForestRegressor(
```

```
    n_estimators = 10,
    criterion    = 'squared_error',
    max_depth    = None,
    max_features = 1,
    oob_score    = False,
    n_jobs       = -1,
    random_state = 123
)
rf_base.fit(X_train, y_train)
print('  Modelo base entrenado')
```

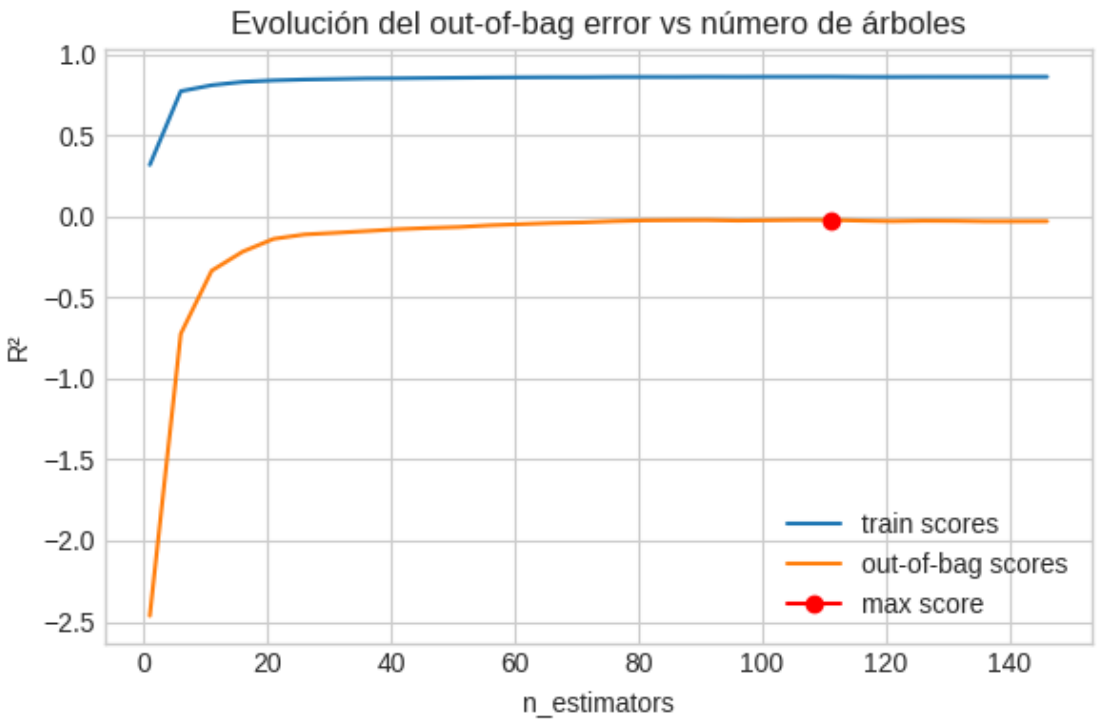
```
modelo base
 Modelo base entrenado
```

[11]:
```
# Predicción y evaluación del modelo base
y_pred_base  = rf_base.predict(X_test)
metricas_base = evaluar_modelo('Random Forest BASE', y_test, y_pred_base)

# Cross-validation 5-fold
cv_r2   = cross_val_score(rf_base, X, y, cv=5, scoring='r2', n_jobs=-1)
cv_rmse = cross_val_score(rf_base, X, y, cv=5,
                          scoring='neg_root_mean_squared_error', n_jobs=-1)
print(f'\n  CV R²   (5-fold): {cv_r2.mean():.4f} ± {cv_r2.std():.4f}')
print(f'  CV RMSE (5-fold): {(-cv_rmse).mean():.4f} ± {(-cv_rmse).std():.4f}')
```

```
==================================================
   Random Forest BASE
==================================================
  RMSE    : 2.9770  ← error en puntos de estrés
  MAE     : 2.4796  ← error absoluto medio
  MAPE    : 83.69%  ← error porcentual
  R²      : -0.1430  ← -14.3% varianza explicada
  R² adj  : -0.2034  ← penaliza por nº features

  CV R²   (5-fold): -0.1119 ± 0.0334
  CV RMSE (5-fold): 3.0576 ± 0.0689
```

[12]:
```
# Validación con Out-of-Bag error
warnings.filterwarnings('ignore')
train_scores = []
oob_scores   = []

estimator_range = range(1, 150, 5)

for n_estimators in estimator_range:
```

```
    modelo = RandomForestRegressor(
        n_estimators = n_estimators,
        criterion    = 'squared_error',
        max_depth    = None,
        max_features = 1,
        oob_score    = True,
        n_jobs       = -1,
        random_state = 123
    )
    modelo.fit(X_train, y_train)
    train_scores.append(modelo.score(X_train, y_train))
    oob_scores.append(modelo.oob_score_)

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(estimator_range, train_scores, label='train scores')
ax.plot(estimator_range, oob_scores,   label='out-of-bag scores')
ax.plot(estimator_range[np.argmax(oob_scores)], max(oob_scores),
        marker='o', color='red', label='max score')
ax.set_ylabel('R²')
ax.set_xlabel('n_estimators')
ax.set_title('Evolución del out-of-bag error vs número de árboles')
ax.legend()
plt.tight_layout()
plt.show()
print(f'Valor óptimo de n_estimators (OOB): {estimator_range[np.
 ↪argmax(oob_scores)]}')
warnings.filterwarnings('default')
```

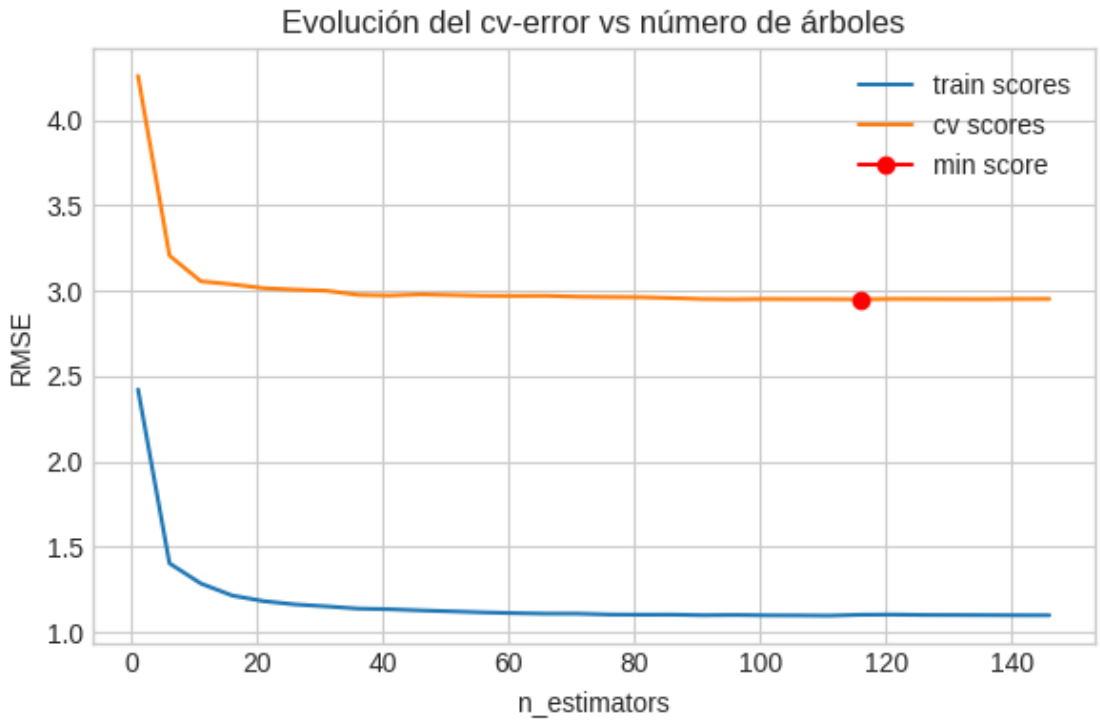



```
Valor óptimo de n_estimators (OOB): 111
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

[13]:
```python
# Validación con k-cross-validation y RMSE
train_scores_cv = []
cv_scores       = []

for n_estimators in estimator_range:
    modelo = RandomForestRegressor(
        n_estimators = n_estimators,
        criterion    = 'squared_error',
        max_depth    = None,
        max_features = 1,
        oob_score    = False,
        n_jobs       = -1,
        random_state = 123
    )
    modelo.fit(X_train, y_train)
    predicciones = modelo.predict(X=X_train)
```

```
    rmse = np.sqrt(mean_squared_error(y_true=y_train, y_pred=predicciones))
    train_scores_cv.append(rmse)

    scores = cross_val_score(
        estimator = modelo, X=X_train, y=y_train,
        scoring   = 'neg_root_mean_squared_error', cv=5
    )
    cv_scores.append(-1 * scores.mean())

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(estimator_range, train_scores_cv, label='train scores')
ax.plot(estimator_range, cv_scores,       label='cv scores')
ax.plot(estimator_range[np.argmin(cv_scores)], min(cv_scores),
        marker='o', color='red', label='min score')
ax.set_ylabel('RMSE')
ax.set_xlabel('n_estimators')
ax.set_title('Evolución del cv-error vs número de árboles')
ax.legend()
plt.tight_layout()
plt.show()
print(f'Valor óptimo de n_estimators (CV): {estimator_range[np.
 ↪argmin(cv_scores)]}')
```




```
Valor óptimo de n_estimators (CV): 116
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

1.6 Sección 6 — Optimización con GridSearchCV

[14]:
```python
# ============================================================================
#  SECCIÓN 6 - OPTIMIZACIÓN CON GridSearchCV
# ============================================================================
print('\n' + '='*55)
print('  OPTIMIZACIÓN DE HIPERPARÁMETROS')
print('='*55)
print('  Estrategia : GridSearchCV (búsqueda exhaustiva)')
print('  CV folds   : 5')
print('  Scoring    : neg_root_mean_squared_error')
print('\n Ejecutando GridSearch (puede tardar 2-3 min)...')

param_grid = {
    'n_estimators'     : [100, 200, 300],
    'max_depth'        : [None, 10, 20],
    'max_features'     : ['sqrt', 'log2', 0.5],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf' : [1, 2, 4],
    'bootstrap'        : [True]
}

n_combos = 1
for v in param_grid.values():
    n_combos *= len(v)
print(f'  Combinaciones a evaluar: {n_combos} × 5 folds = {n_combos*5} fits')

rf_grid = GridSearchCV(
    estimator  = RandomForestRegressor(random_state=SEED, n_jobs=-1),
    param_grid = param_grid,
    cv         = 5,
    scoring    = 'neg_root_mean_squared_error',
    n_jobs     = -1,
    verbose    = 1,
    refit      = True
)
rf_grid.fit(X_train, y_train)

print('\n Mejores hiperparámetros:')
for param, valor in rf_grid.best_params_.items():
    print(f'   {param:<22}: {valor}')
```

```
print(f'\n   Mejor RMSE (CV): {-rf_grid.best_score_:.4f} puntos de estrés')
```

```
============================================================
  OPTIMIZACIÓN DE HIPERPARÁMETROS
============================================================
  Estrategia : GridSearchCV (búsqueda exhaustiva)
  CV folds   : 5
  Scoring    : neg_root_mean_squared_error

  Ejecutando GridSearch (puede tardar 2-3 min)…
  Combinaciones a evaluar: 243 × 5 folds = 1215 fits
Fitting 5 folds for each of 243 candidates, totalling 1215 fits

  Mejores hiperparámetros:
    bootstrap             : True
    max_depth             : 10
    max_features          : 0.5
    min_samples_leaf      : 4
    min_samples_split     : 10
    n_estimators          : 100

    Mejor RMSE (CV): 2.9177 puntos de estrés
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

1.7 Sección 7 — Evaluación Final del Modelo Optimizado

[15]:
```
# ============================================================================
#  SECCIÓN 7 - EVALUACIÓN FINAL DEL MODELO OPTIMIZADO
# ============================================================================

best_rf      = rf_grid.best_estimator_
y_pred_opt   = best_rf.predict(X_test)
metricas_opt = evaluar_modelo('Random Forest OPTIMIZADO', y_test, y_pred_opt)

# CV del modelo optimizado
cv_r2_opt = cross_val_score(best_rf, X, y, cv=5, scoring='r2', n_jobs=-1)
print(f'\n  CV R² optimizado (5-fold): {cv_r2_opt.mean():.4f} ± {cv_r2_opt.
  ↪std():.4f}')

# Tabla comparativa
print('\n' + '='*55)
print('  COMPARATIVA: BASE vs OPTIMIZADO')
```

```python
print('='*55)
print(f"  {'Métrica':<10} {'Base':>10} {'Optimizado':>12} {'Δ Mejora':>12}")
print('  ' + '-'*48)
for m in ['RMSE', 'MAE', 'MAPE', 'R2', 'R2_adj']:
    base = metricas_base[m]
    opt  = metricas_opt[m]
    if m in ['R2', 'R2_adj']:
        delta = f'+{(opt-base)*100:.2f}pp'
    else:
        delta = f'-{((base-opt)/base)*100:.1f}%' if base > opt else␣
↪f'+{((opt-base)/base)*100:.1f}%'
    print(f'  {m:<10} {base:>10.4f} {opt:>12.4f} {delta:>12}')
```

```
==================================================
   Random Forest OPTIMIZADO
==================================================
  RMSE    : 2.8387  ← error en puntos de estrés
  MAE     : 2.3924  ← error absoluto medio
  MAPE    : 84.47%  ← error porcentual
  R²      : -0.0392  ← -3.9% varianza explicada
  R² adj  : -0.0942  ← penaliza por nº features
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)


  CV R² optimizado (5-fold): -0.0077 ± 0.0281

=========================================================
  COMPARATIVA: BASE vs OPTIMIZADO
=========================================================
  Métrica          Base    Optimizado     Δ Mejora
  -------------------------------------------------
  RMSE           2.9770       2.8387        -4.6%
  MAE            2.4796       2.3924        -3.5%
  MAPE          83.6928      84.4699        +0.9%
  R2            -0.1430      -0.0392     +10.38pp
  R2_adj        -0.2034      -0.0942     +10.93pp
```

1.8 Sección 8 — Importancia de Variables

[16]:
```
# ============================================================================
#  SECCIÓN 8 - IMPORTANCIA DE VARIABLES
# ============================================================================

# 8.1 Impurity-based (Gini)
imp_gini = pd.Series(best_rf.feature_importances_, index=FEATURES).
 ↪sort_values(ascending=False)

# 8.2 Permutation Importance (más confiable)
print('\n Calculando Permutation Importance...')
perm_imp = permutation_importance(best_rf, X_test, y_test,
                                  n_repeats=30, random_state=SEED, n_jobs=-1)
imp_perm = pd.Series(perm_imp.importances_mean, index=FEATURES).
 ↪sort_values(ascending=False)

print('\n  IMPORTANCIA DE FEATURES (Top 5):')
print(f"  {'Feature':<30} {'Gini':>8} {'Permut.':>10}")
print('  ' + '-'*50)
for feat in imp_gini.index[:5]:
    print(f'  {feat:<30} {imp_gini[feat]:>8.4f} {imp_perm[feat]:>10.4f}')
```

```
 Calculando Permutation Importance…
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)


  IMPORTANCIA DE FEATURES (Top 5):
  Feature                            Gini    Permut.
  --------------------------------------------------
  daily_social_media_hours         0.1501     0.0072
  academic_performance             0.1398    -0.0108
  sleep_hours                      0.1345     0.0052
  screen_time_before_sleep         0.1041    -0.0063
  physical_activity                0.0973    -0.0114
```

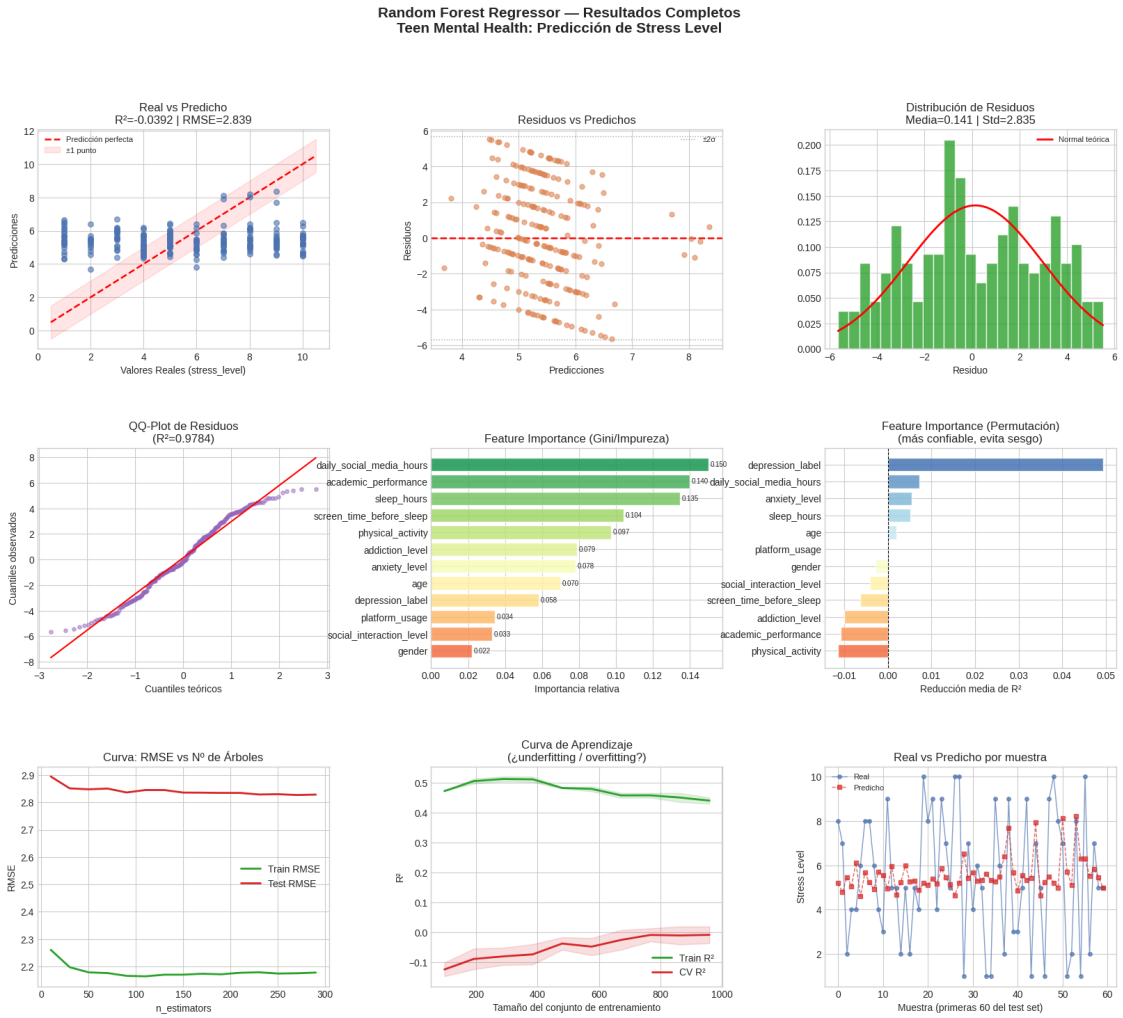
1.9 Sección 9 — Visualizaciones Finales

[17]:
```
# ============================================================================
#  SECCIÓN 9 - VISUALIZACIONES FINALES
# ============================================================================

fig = plt.figure(figsize=(20, 16))
```

```
gs  = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)
fig.suptitle('Random Forest Regressor - Resultados Completos\nTeen Mental␣
 ↪Health: Predicción de Stress Level',
             fontsize=15, fontweight='bold', y=0.99)

# (a) Real vs Predicho
ax1 = fig.add_subplot(gs[0, 0])
ax1.scatter(y_test, y_pred_opt, alpha=0.6, color='#4C72B0', s=30, zorder=3)
lim = [0.5, 10.5]
ax1.plot(lim, lim, 'r--', linewidth=1.8, label='Predicción perfecta', zorder=2)
ax1.fill_between(lim, [l-1 for l in lim], [l+1 for l in lim],
                 alpha=0.1, color='red', label='±1 punto')
ax1.set_xlabel('Valores Reales (stress_level)')
ax1.set_ylabel('Predicciones')
ax1.set_title(f"Real vs Predicho\nR²={metricas_opt['R2']:.4f} |␣
 ↪RMSE={metricas_opt['RMSE']:.3f}")
ax1.legend(fontsize=8)

# (b) Residuos vs Predichos
ax2 = fig.add_subplot(gs[0, 1])
residuos = y_test.values - y_pred_opt
ax2.scatter(y_pred_opt, residuos, alpha=0.6, color='#dd8452', s=25, zorder=3)
ax2.axhline(0, color='red', linewidth=1.8, linestyle='--')
ax2.axhline( residuos.std()*2, color='gray', linewidth=1, linestyle=':',␣
 ↪alpha=0.7)
ax2.axhline(-residuos.std()*2, color='gray', linewidth=1, linestyle=':',␣
 ↪alpha=0.7, label='±2 ')
ax2.set_xlabel('Predicciones')
ax2.set_ylabel('Residuos')
ax2.set_title('Residuos vs Predichos')
ax2.legend(fontsize=8)

# (c) Distribución de residuos
ax3 = fig.add_subplot(gs[0, 2])
ax3.hist(residuos, bins=25, color='#2ca02c', edgecolor='white', alpha=0.8,␣
 ↪density=True)
xr = np.linspace(residuos.min(), residuos.max(), 200)
ax3.plot(xr, stats.norm.pdf(xr, residuos.mean(), residuos.std()),
         'r-', linewidth=2, label='Normal teórica')
ax3.set_title(f'Distribución de Residuos\nMedia={residuos.mean():.3f} |␣
 ↪Std={residuos.std():.3f}')
ax3.set_xlabel('Residuo')
ax3.legend(fontsize=8)

# (d) QQ-Plot de residuos
ax4 = fig.add_subplot(gs[1, 0])
```

```
(osm2, osr2), (slope2, intercept2, r2_qq) = stats.probplot(residuos,␣
 ↪dist='norm')
ax4.scatter(osm2, osr2, alpha=0.5, s=15, color='#9467bd')
ax4.plot(osm2, slope2*np.array(osm2)+intercept2, 'r-', linewidth=1.5)
ax4.set_title(f'QQ-Plot de Residuos\n(R²={r2_qq**2:.4f})')
ax4.set_xlabel('Cuantiles teóricos')
ax4.set_ylabel('Cuantiles observados')

# (e) Feature Importance - Gini
ax5 = fig.add_subplot(gs[1, 1])
imp_gini_sorted = imp_gini.sort_values(ascending=True)
colors_g = plt.cm.RdYlGn(np.linspace(0.2, 0.9, len(imp_gini_sorted)))
ax5.barh(imp_gini_sorted.index, imp_gini_sorted.values, color=colors_g, alpha=0.
 ↪85, edgecolor='white')
ax5.set_title('Feature Importance (Gini/Impureza)')
ax5.set_xlabel('Importancia relativa')
for i, (feat, val) in enumerate(imp_gini_sorted.items()):
    ax5.text(val + 0.001, i, f'{val:.3f}', va='center', fontsize=7)

# (f) Feature Importance - Permutation
ax6 = fig.add_subplot(gs[1, 2])
imp_perm_sorted = imp_perm.sort_values(ascending=True)
colors_p = plt.cm.RdYlBu(np.linspace(0.2, 0.9, len(imp_perm_sorted)))
ax6.barh(imp_perm_sorted.index, imp_perm_sorted.values, color=colors_p, alpha=0.
 ↪85, edgecolor='white')
ax6.axvline(0, color='black', linewidth=0.8, linestyle='--')
ax6.set_title('Feature Importance (Permutación)\n(más confiable, evita sesgo)')
ax6.set_xlabel('Reducción media de R²')

# (g) Curva RMSE vs n_estimators
ax7 = fig.add_subplot(gs[2, 0])
n_range = range(10, 310, 20)
rmse_tr, rmse_te = [], []
best_params_sin_n = {k: v for k, v in rf_grid.best_params_.items() if k !=␣
 ↪'n_estimators'}
for n in n_range:
    m = RandomForestRegressor(n_estimators=n, **best_params_sin_n,␣
 ↪random_state=SEED, n_jobs=-1)
    m.fit(X_train, y_train)
    rmse_tr.append(np.sqrt(mean_squared_error(y_train, m.predict(X_train))))
    rmse_te.append(np.sqrt(mean_squared_error(y_test,  m.predict(X_test))))
ax7.plot(n_range, rmse_tr, label='Train RMSE', color='#2ca02c', linewidth=2)
ax7.plot(n_range, rmse_te, label='Test RMSE',  color='#d62728', linewidth=2)
ax7.set_xlabel('n_estimators')
ax7.set_ylabel('RMSE')
ax7.set_title('Curva: RMSE vs Nº de Árboles')
ax7.legend()
```

```
# (h) Curva de aprendizaje (tamaño del dataset)
ax8 = fig.add_subplot(gs[2, 1])
train_sizes, train_sc, val_sc = learning_curve(
    best_rf, X, y, cv=5, scoring='r2',
    train_sizes=np.linspace(0.1, 1.0, 10), n_jobs=-1
)
ax8.plot(train_sizes, train_sc.mean(axis=1), label='Train R²', color='#2ca02c',␣
 ↪linewidth=2)
ax8.fill_between(train_sizes,
                 train_sc.mean(axis=1) - train_sc.std(axis=1),
                 train_sc.mean(axis=1) + train_sc.std(axis=1),
                 alpha=0.15, color='#2ca02c')
ax8.plot(train_sizes, val_sc.mean(axis=1), label='CV R²', color='#d62728',␣
 ↪linewidth=2)
ax8.fill_between(train_sizes,
                 val_sc.mean(axis=1) - val_sc.std(axis=1),
                 val_sc.mean(axis=1) + val_sc.std(axis=1),
                 alpha=0.15, color='#d62728')
ax8.set_xlabel('Tamaño del conjunto de entrenamiento')
ax8.set_ylabel('R²')
ax8.set_title('Curva de Aprendizaje\n(¿underfitting / overfitting?)')
ax8.legend()

# (i) Predicciones vs índice
ax9 = fig.add_subplot(gs[2, 2])
idx_plot = np.arange(60)  # primeras 60 muestras
ax9.plot(idx_plot, y_test.values[:60],  'o-', color='#4C72B0', alpha=0.7,␣
 ↪markersize=4, linewidth=1, label='Real')
ax9.plot(idx_plot, y_pred_opt[:60], 's--', color='#d62728', alpha=0.7,␣
 ↪markersize=4, linewidth=1, label='Predicho')
ax9.set_xlabel('Muestra (primeras 60 del test set)')
ax9.set_ylabel('Stress Level')
ax9.set_title('Real vs Predicho por muestra')
ax9.legend(fontsize=8)

plt.savefig('02_resultados_completos.png', dpi=150, bbox_inches='tight')
plt.show()
print('  Resultados guardados → 02_resultados_completos.png')
```




```
Resultados guardados → 02_resultados_completos.png
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

1.10 Sección 10 — Predicciones e Interpretación

[18]:
```
# ================================================================================
#  SECCIÓN 10A - RESUMEN EJECUTIVO + TEST DE NORMALIDAD DE RESIDUOS
# ================================================================================

stat_sw, p_sw = stats.shapiro(residuos[:50])
```

```
stat_ks, p_ks = stats.kstest(residuos, 'norm', args=(residuos.mean(), residuos.
 ↪std()))

print('\n' + '='*55)
print('  TEST DE NORMALIDAD DE RESIDUOS')
print('='*55)
print(f'  Shapiro-Wilk : W={stat_sw:.4f}, p={p_sw:.4f}  '
      f'{" Normal" if p_sw > 0.05 else "  No normal"}')
print(f'  KS Test      : D={stat_ks:.4f}, p={p_ks:.4f}  '
      f'{" Normal" if p_ks > 0.05 else "  No normal"}')

print('\n' + '='*55)
print('   RESUMEN EJECUTIVO')
print('='*55)
print(f'  Dataset           : Teen Mental Health ({len(df)} obs,␣
 ↪{len(FEATURES)} features)')
print(f'  Variable objetivo : stress_level - escala 1-10')
print(f'  Algoritmo         : Random Forest Regressor')
print(f'  Split             : 80% train / 20% test')
print(f'  Optimización      : GridSearchCV (5-fold CV)')
print()
print(f'    Métricas finales (test set)  ')
print(f'  RMSE   : {metricas_opt["RMSE"]:.4f} puntos de estrés')
print(f'  MAE    : {metricas_opt["MAE"]:.4f} puntos de estrés')
print(f'  MAPE   : {metricas_opt["MAPE"]:.2f}%')
print(f'  R²     : {metricas_opt["R2"]:.4f}')
print(f'  R² adj : {metricas_opt["R2_adj"]:.4f}')
print()
print(f'    Mejores hiperparámetros  ')
for k, v in rf_grid.best_params_.items():
    print(f'  {k:<22}: {v}')
print()
print(f'    Top 3 features más importantes (Gini)  ')
for i, (feat, val) in enumerate(imp_gini.head(3).items(), 1):
    print(f'  {i}. {feat:<30}: {val:.4f} (Gini) | {imp_perm[feat]:.4f} (Permut.
 ↪)')
print('='*55)
```

```
=======================================================
  TEST DE NORMALIDAD DE RESIDUOS
=======================================================
  Shapiro-Wilk : W=0.9675, p=0.1819    Normal
  KS Test      : D=0.0543, p=0.4620    Normal

=======================================================
   RESUMEN EJECUTIVO
```

```
=======================================================
  Dataset           : Teen Mental Health (1200 obs, 12 features)
  Variable objetivo : stress_level - escala 1-10
  Algoritmo         : Random Forest Regressor
  Split             : 80% train / 20% test
  Optimización      : GridSearchCV (5-fold CV)

    Métricas finales (test set)
  RMSE   : 2.8387 puntos de estrés
  MAE    : 2.3924 puntos de estrés
  MAPE   : 84.47%
  R²     : -0.0392
  R² adj : -0.0942

    Mejores hiperparámetros
  bootstrap             : True
  max_depth             : 10
  max_features          : 0.5
  min_samples_leaf      : 4
  min_samples_split     : 10
  n_estimators          : 100

    Top 3 features más importantes (Gini)
  1. daily_social_media_hours      : 0.1501 (Gini) | 0.0072 (Permut.)
  2. academic_performance          : 0.1398 (Gini) | -0.0108 (Permut.)
  3. sleep_hours                   : 0.1345 (Gini) | 0.0052 (Permut.)
=======================================================
```

[19]:
```python
# ============================================================================
#  SECCIÓN 10B - FUNCIÓN DE PREDICCIÓN CON INTERVALO DE CONFIANZA
# ============================================================================

def predecir_estres(modelo, datos_entrada, nombres_features):
    """
    Predice el nivel de estrés con:
      - Predicción puntual (media de los árboles)
      - Intervalo de confianza empírico (percentil 5-95)
      - Nivel de confianza del modelo

    Entrada:
    - datos_entrada : dict o DataFrame con valores ya codificados
    - nombres_features: lista con los nombres de las columnas

    Notas de codificación:
      gender                 : male=1, female=0
      platform_usage         : Both=0, Instagram=1, TikTok=2
      social_interaction_level: high=0, low=1, medium=2
```

```
    """
    if isinstance(datos_entrada, dict):
        df_entrada = pd.DataFrame([datos_entrada])
    else:
        df_entrada = datos_entrada.copy()

    df_entrada = df_entrada[nombres_features]

    predicciones_arboles = np.array([
        arbol.predict(df_entrada) for arbol in modelo.estimators_
    ])

    pred_media = predicciones_arboles.mean(axis=0)
    pred_std   = predicciones_arboles.std(axis=0)
    pred_p5    = np.percentile(predicciones_arboles, 5,  axis=0)
    pred_p95   = np.percentile(predicciones_arboles, 95, axis=0)
    pred_p25   = np.percentile(predicciones_arboles, 25, axis=0)
    pred_p75   = np.percentile(predicciones_arboles, 75, axis=0)

    cv_modelo = (pred_std / pred_media) * 100
    confianza = np.where(cv_modelo < 5,  '  Alta',
                np.where(cv_modelo < 15, '  Media', '  Baja'))

    resultados = pd.DataFrame({
        'Stress estimado (1-10)' : pred_media.round(2),
        'IC 90% inferior'        : pred_p5.round(2),
        'IC 90% superior'        : pred_p95.round(2),
        'IQR Q25'                : pred_p25.round(2),
        'IQR Q75'                : pred_p75.round(2),
        'Std entre árboles'      : pred_std.round(4),
        'CV modelo (%)'          : cv_modelo.round(2),
        'Confianza'              : confianza
    })
    return resultados, predicciones_arboles

print('  Función predecir_estres() definida')
print()
print('  Notas de codificación para las variables categóricas:')
for col, le in encoders.items():
    print(f'  {col:<30}: {dict(zip(le.classes_, le.transform(le.classes_)))}')
```

```
  Función predecir_estres() definida

  Notas de codificación para las variables categóricas:
  gender                         : {'female': np.int64(0), 'male': np.int64(1)}
  platform_usage                 : {'Both': np.int64(0), 'Instagram':
np.int64(1), 'TikTok': np.int64(2)}
```

```
  social_interaction_level      : {'high': np.int64(0), 'low': np.int64(1),
'medium': np.int64(2)}

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

[20]:

```python
# ============================================================================
#  CASO 1 - PREDICCIÓN INDIVIDUAL (un adolescente)
# ============================================================================

print('\n' + '='*60)
print('  CASO 1: PREDICCIÓN INDIVIDUAL - UN ADOLESCENTE')
print('='*60)

# Codificación recordatorio:
#   gender: male=1, female=0
#   platform_usage: Both=0, Instagram=1, TikTok=2
#   social_interaction_level: high=0, low=1, medium=2

adolescente_nuevo = {
    'age'                     : 16,
    'gender'                  : 0,    # female
    'daily_social_media_hours' : 6.5,  # muchas horas en redes
    'platform_usage'          : 2,    # TikTok
    'sleep_hours'             : 5.5,  # poco sueño
    'screen_time_before_sleep' : 2.5,  # pantalla antes de dormir
    'academic_performance'    : 2.8,  # rendimiento bajo
    'physical_activity'       : 0.5,  # poca actividad física
    'social_interaction_level' : 1,    # low
    'anxiety_level'           : 7,    # alta ansiedad
    'addiction_level'         : 8,    # alta adicción a redes
    'depression_label'        : 0
}

resultado_1, dist_arboles_1 = predecir_estres(best_rf, adolescente_nuevo,␣
 ↪FEATURES)

print('\n   Perfil del adolescente:')
perfil_legible = {
    'Edad': 16, 'Género': 'Femenino', 'Horas en redes/día': 6.5,
    'Plataforma': 'TikTok', 'Horas de sueño': 5.5,
    'Pantalla antes de dormir': 2.5, 'Rendimiento académico': 2.8,
    'Actividad física/día': 0.5, 'Interacción social': 'Baja',
    'Nivel de ansiedad': 7, 'Adicción a redes': 8
}
```

```python
for k, v in perfil_legible.items():
    print(f'     {k:<28}: {v}')

print(f"\n    PREDICCIÓN DE ESTRÉS:")
print(f"     Nivel estimado    : {resultado_1['Stress estimado (1-10)'].iloc[0]:.
 ↪2f} / 10")
print(f"\n    INTERVALO DE CONFIANZA (90%):")
print(f"     Límite inferior   : {resultado_1['IC 90% inferior'].iloc[0]:.2f}")
print(f"     Límite superior   : {resultado_1['IC 90% superior'].iloc[0]:.2f}")
print(f"\n    CONFIABILIDAD:")
print(f"     Std entre árboles: {resultado_1['Std entre árboles'].iloc[0]:.4f}")
print(f"     CV del modelo     : {resultado_1['CV modelo (%)'].iloc[0]:.2f}%")
print(f"     Nivel confianza   : {resultado_1['Confianza'].iloc[0]}")
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(


============================================================
  CASO 1: PREDICCIÓN INDIVIDUAL - UN ADOLESCENTE
============================================================

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
```

```
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
```

```
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
```

```
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
```

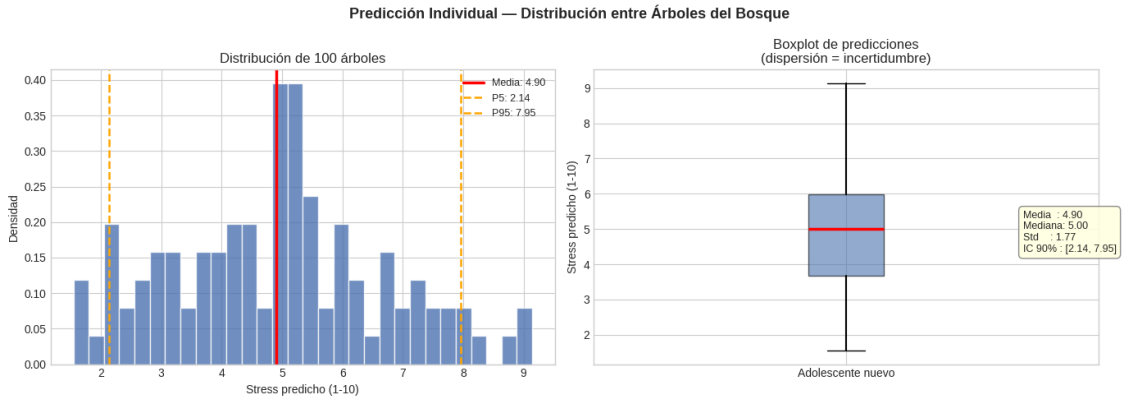
```
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(


    Perfil del adolescente:
      Edad                        : 16
      Género                      : Femenino
      Horas en redes/día          : 6.5
      Plataforma                  : TikTok
      Horas de sueño              : 5.5
      Pantalla antes de dormir    : 2.5
      Rendimiento académico       : 2.8
      Actividad física/día        : 0.5
      Interacción social          : Baja
      Nivel de ansiedad           : 7
      Adicción a redes            : 8

    PREDICCIÓN DE ESTRÉS:
      Nivel estimado   : 4.90 / 10

    INTERVALO DE CONFIANZA (90%):
      Límite inferior  : 2.14
      Límite superior  : 7.95

    CONFIABILIDAD:
      Std entre árboles: 1.7676
      CV del modelo    : 36.04%
      Nivel confianza  :   Baja

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

[21]:
```
# Visualización: distribución de predicciones entre árboles
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Predicción Individual - Distribución entre Árboles del Bosque',
             fontsize=13, fontweight='bold')

pred_flat = dist_arboles_1[:, 0]

# Histograma
axes[0].hist(pred_flat, bins=30, color='#4C72B0', edgecolor='white', alpha=0.8,␣
 ↪density=True)
axes[0].axvline(pred_flat.mean(), color='red', linewidth=2.5, linestyle='-',
                label=f'Media: {pred_flat.mean():.2f}')
axes[0].axvline(np.percentile(pred_flat, 5),  color='orange', linewidth=1.8,
                linestyle='--', label=f'P5: {np.percentile(pred_flat,5):.2f}')
axes[0].axvline(np.percentile(pred_flat, 95), color='orange', linewidth=1.8,
                linestyle='--', label=f'P95: {np.percentile(pred_flat,95):.2f}')
axes[0].set_xlabel('Stress predicho (1-10)')
axes[0].set_ylabel('Densidad')
axes[0].set_title(f'Distribución de {len(pred_flat)} árboles')
axes[0].legend(fontsize=9)

# Boxplot
axes[1].boxplot(pred_flat, vert=True, patch_artist=True,
                boxprops=dict(facecolor='#4C72B0', alpha=0.6),
                medianprops=dict(color='red', linewidth=2.5),
                whiskerprops=dict(linewidth=1.5),
                flierprops=dict(marker='o', markersize=4, alpha=0.4))
axes[1].set_ylabel('Stress predicho (1-10)')
axes[1].set_title('Boxplot de predicciones\n(dispersión = incertidumbre)')
axes[1].set_xticks([1])
axes[1].set_xticklabels(['Adolescente nuevo'])

stats_text = (f'Media  : {pred_flat.mean():.2f}\n'
              f'Mediana: {np.median(pred_flat):.2f}\n'
              f'Std    : {pred_flat.std():.2f}\n'
              f'IC 90% : [{np.percentile(pred_flat,5):.2f}, '
              f'{np.percentile(pred_flat,95):.2f}]')
axes[1].text(1.35, pred_flat.mean(), stats_text, fontsize=9,
             bbox=dict(boxstyle='round,pad=0.4', facecolor='lightyellow',
                       edgecolor='gray', alpha=0.9), va='center')

plt.tight_layout()
plt.savefig('03_prediccion_individual.png', dpi=150, bbox_inches='tight')
plt.show()
print('  Gráfico guardado → 03_prediccion_individual.png')
```

```
/tmp/ipykernel_18454/707914608.py:22: PendingDeprecationWarning: vert: bool will
be deprecated in a future version. Use orientation: {'vertical', 'horizontal'}
instead.
  axes[1].boxplot(pred_flat, vert=True, patch_artist=True,
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```




```
Gráfico guardado → 03_prediccion_individual.png
```

[22]:
```python
# ============================================================================
#  CASO 2 - COMPARACIÓN DE PERFILES (escenarios de riesgo)
# ============================================================================

print('\n' + '='*60)
print('  CASO 2: COMPARACIÓN DE PERFILES DE RIESGO')
print('='*60)

# gender: male=1, female=0
# platform_usage: Both=0, Instagram=1, TikTok=2
# social_interaction_level: high=0, low=1, medium=2

escenarios = pd.DataFrame({
    'age'                   : [14,   17,   16,   18],
    'gender'                : [0,    1,    0,    1],
    'daily_social_media_hours' : [8.0,  1.5,  5.0,  3.0],
```

```
    'platform_usage'           : [2,    1,    0,    1],
    'sleep_hours'              : [4.5,  8.0,  6.0,  7.5],
    'screen_time_before_sleep' : [3.0,  0.5,  1.5,  1.0],
    'academic_performance'     : [2.1,  3.8,  3.0,  3.5],
    'physical_activity'        : [0.2,  2.0,  1.0,  1.5],
    'social_interaction_level' : [1,    0,    2,    0],
    'anxiety_level'            : [9,    2,    5,    3],
    'addiction_level'          : [9,    1,    5,    3],
    'depression_label'         : [1,    0,    0,    0]
})

nombres_perfiles = [
    '  Alto Riesgo\n(mucho TikTok, sin dormir)',
    '  Bajo Riesgo\n(equilibrado, buen sueño)',
    '  Riesgo Medio\n(uso moderado)',
    '  Bajo-Medio\n(activo, poca ansiedad)'
]

resultados_esc, dist_esc = predecir_estres(best_rf, escenarios, FEATURES)
resultados_esc.index = nombres_perfiles

print('\n  Resultados por perfil:')
cols_ver = ['Stress estimado (1-10)', 'IC 90% inferior', 'IC 90% superior', 'CV␣
 ↪modelo (%)', 'Confianza']
print(resultados_esc[cols_ver].to_string())

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Predicción por Perfiles - Comparativa de Riesgo de Estrés',
             fontsize=13, fontweight='bold')

precios  = resultados_esc['Stress estimado (1-10)'].values
ic_inf   = resultados_esc['IC 90% inferior'].values
ic_sup   = resultados_esc['IC 90% superior'].values
yerr_inf = precios - ic_inf
yerr_sup = ic_sup - precios
colores  = ['#d62728', '#2ca02c', '#ff7f0e', '#4C72B0']
etiquetas_cortas = ['Alto Riesgo', 'Bajo Riesgo', 'Riesgo Medio', 'Bajo-Medio']

bars = axes[0].bar(etiquetas_cortas, precios, color=colores, alpha=0.8,␣
 ↪edgecolor='white',
                   yerr=[yerr_inf, yerr_sup], capsize=8,
                   error_kw={'elinewidth': 2, 'ecolor': 'black', 'alpha': 0.7})
axes[0].set_ylabel('Nivel de estrés predicho (1-10)')
axes[0].set_title('Estrés estimado por perfil\n(barras de error = IC 90%)')
axes[0].set_ylim(0, 11)
for bar, precio in zip(bars, precios):
```

```
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
                 f'{precio:.1f}', ha='center', va='bottom', fontweight='bold',␣
 ↪fontsize=11)

axes[1].boxplot(
    [dist_esc[:, i] for i in range(len(escenarios))],
    labels=etiquetas_cortas, patch_artist=True,
    boxprops=dict(alpha=0.7),
    medianprops=dict(color='black', linewidth=2)
)
for patch, color in zip(axes[1].patches, colores):
    patch.set_facecolor(color)
axes[1].set_ylabel('Stress predicho (1-10)')
axes[1].set_title('Distribución de predicciones\npor perfil (todos los␣
 ↪árboles)')

plt.tight_layout()
plt.savefig('04_prediccion_escenarios.png', dpi=150, bbox_inches='tight')
plt.show()
print('  Gráfico guardado → 04_prediccion_escenarios.png')
```

```
============================================================
  CASO 2: COMPARACIÓN DE PERFILES DE RIESGO
============================================================

  Resultados por perfil:
                                          Stress estimado (1-10)  IC 90%
inferior  IC 90% superior  CV modelo (%) Confianza
  Alto Riesgo\n(mucho TikTok, sin dormir)                    7.65
3.16             9.67          23.86     Baja
  Bajo Riesgo\n(equilibrado, buen sueño)                     4.96
1.93             8.81          45.77     Baja
  Riesgo Medio\n(uso moderado)                               5.98
3.82             8.56          23.84     Baja
  Bajo-Medio\n(activo, poca ansiedad)                        5.17
2.83             7.73          28.29     Baja
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
```

```
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

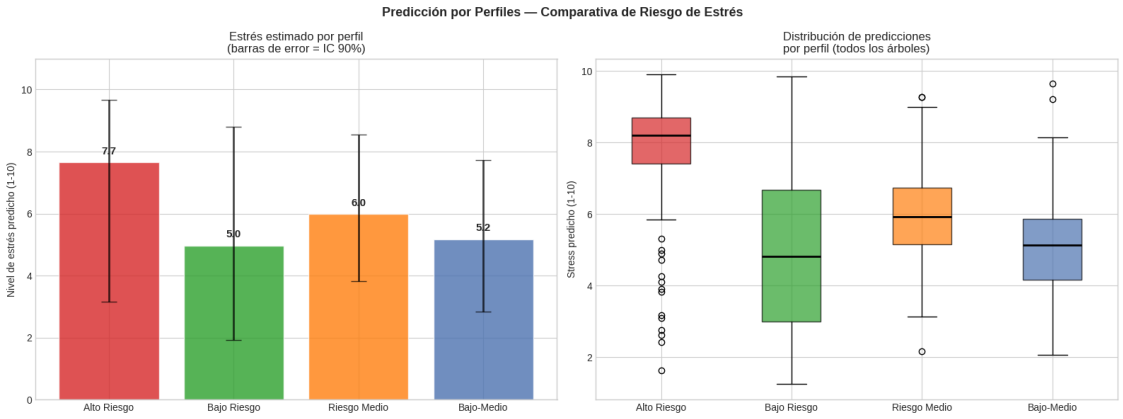
```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
```

```
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732:
UserWarning: X has feature names, but DecisionTreeRegressor was fitted without
feature names
  warnings.warn(
/tmp/ipykernel_18454/3535303013.py:65: MatplotlibDeprecationWarning: The
'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib
3.9; support for the old name will be dropped in 3.11.
  axes[1].boxplot(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```




```
Gráfico guardado → 04_prediccion_escenarios.png
```

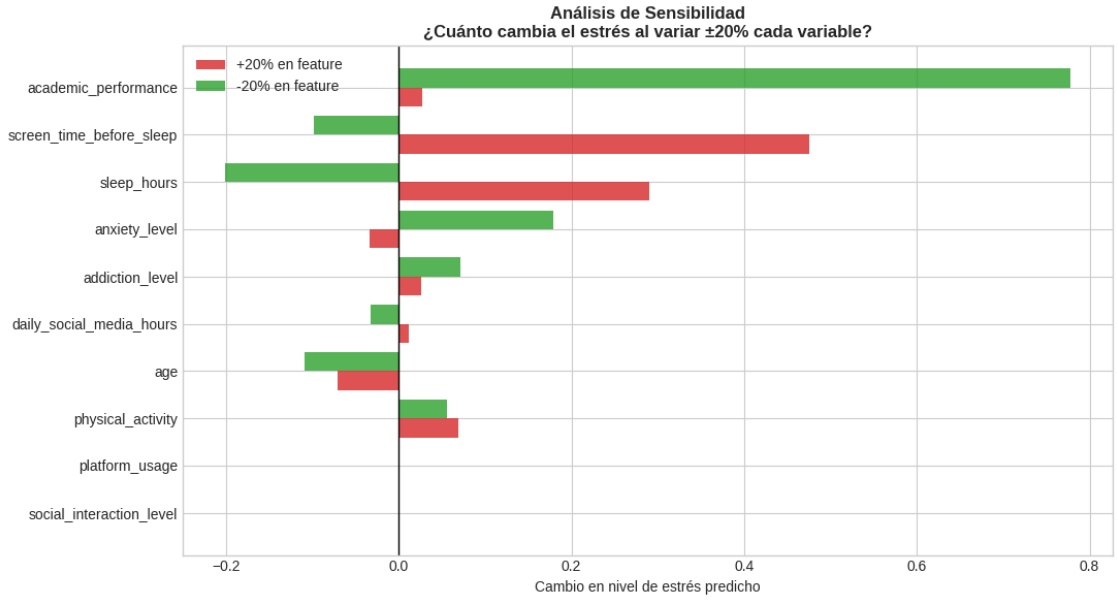
[23]:
```python
# ============================================================================
#  CASO 3 - ANÁLISIS DE SENSIBILIDAD
#  ¿Qué variable impacta más en el nivel de estrés?
# ============================================================================

print('\n' + '='*60)
print('  CASO 3: ANÁLISIS DE SENSIBILIDAD')
print('  ¿Cuánto cambia el estrés al variar ±20% cada feature?')
print('='*60)
```

```
base_vals    = pd.Series(adolescente_nuevo)
sensibilidad = {}

for feat in FEATURES:
    val_original = base_vals[feat]
    if val_original == 0:
        continue  # evitar división por cero

    caso_up = adolescente_nuevo.copy()
    caso_up[feat] = val_original * 1.20
    pred_up = best_rf.predict(pd.DataFrame([caso_up])[FEATURES])[0]

    caso_dn = adolescente_nuevo.copy()
    caso_dn[feat] = val_original * 0.80
    pred_dn = best_rf.predict(pd.DataFrame([caso_dn])[FEATURES])[0]

    pred_base = best_rf.predict(pd.DataFrame([adolescente_nuevo])[FEATURES])[0]

    sensibilidad[feat] = {
        'Δ +20%': round(pred_up - pred_base, 4),
        'Δ -20%': round(pred_dn - pred_base, 4),
        'Rango impacto': round(abs(pred_up - pred_dn), 4)
    }

df_sens = pd.DataFrame(sensibilidad).T.sort_values('Rango impacto',␣
 ↪ascending=False)

print(f'\n  Stress base: {pred_base:.2f} / 10')
print(f"\n  {'Feature':<30} {'Δ +20%':>10} {'Δ -20%':>10} {'Rango':>10}")
print('  ' + '-'*62)
for feat, row in df_sens.iterrows():
    barra = ' ' * int(abs(row['Rango impacto']) * 8)
    print(f"  {feat:<30} {row['Δ +20%']:>+10.4f} {row['Δ -20%']:>+10.4f} ␣
 ↪{row['Rango impacto']:>6.4f}  {barra}")

# Gráfico de sensibilidad
fig, ax = plt.subplots(figsize=(11, 6))
df_sens_sorted = df_sens.sort_values('Rango impacto', ascending=True)
y_pos = range(len(df_sens_sorted))

ax.barh(y_pos, df_sens_sorted['Δ +20%'], height=0.4,
        color='#d62728', alpha=0.8, label='+20% en feature', align='center')
ax.barh([y + 0.4 for y in y_pos], df_sens_sorted['Δ -20%'], height=0.4,
        color='#2ca02c', alpha=0.8, label='-20% en feature', align='center')
ax.axvline(0, color='black', linewidth=1)
ax.set_yticks([y + 0.2 for y in y_pos])
ax.set_yticklabels(df_sens_sorted.index)
```

```
ax.set_xlabel('Cambio en nivel de estrés predicho')
ax.set_title('Análisis de Sensibilidad\n¿Cuánto cambia el estrés al variar ±20%␣
 ↪cada variable?',
             fontsize=12, fontweight='bold')
ax.legend()
plt.tight_layout()
plt.savefig('05_sensibilidad.png', dpi=150, bbox_inches='tight')
plt.show()
print('  Gráfico guardado → 05_sensibilidad.png')
```

```
============================================================
  CASO 3: ANÁLISIS DE SENSIBILIDAD
  ¿Cuánto cambia el estrés al variar ±20% cada feature?
============================================================

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```

```
  Stress base: 4.90 / 10

  Feature                              Δ +20%     Δ -20%      Rango
  ------------------------------------------------------------------
  academic_performance                +0.0276    +0.7779  0.7503
  screen_time_before_sleep            +0.4751    -0.0983  0.5734
  sleep_hours                         +0.2899    -0.2009  0.4908
  anxiety_level                       -0.0338    +0.1792  0.2130
  addiction_level                     +0.0257    +0.0716  0.0459
  daily_social_media_hours            +0.0115    -0.0324  0.0440
  age                                 -0.0713    -0.1089  0.0376
  physical_activity                   +0.0686    +0.0558  0.0128
  platform_usage                      +0.0000    +0.0000  0.0000
  social_interaction_level            +0.0000    +0.0000  0.0000
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
```




```
  Gráfico guardado → 05_sensibilidad.png
```

[24]:
```
# ============================================================================
#  BONUS - BÚSQUEDA BAYESIANA CON OPTUNA
# ============================================================================

!pip install optuna -q
```

```
import optuna
from sklearn.model_selection import RepeatedKFold
optuna.logging.set_verbosity(optuna.logging.WARNING)

def objective(trial):
    params = {
        'n_estimators'     : trial.suggest_int('n_estimators', 100, 800,␣
 ↪step=100),
        'max_depth'        : trial.suggest_int('max_depth', 3, 25),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 50),
        'min_samples_leaf' : trial.suggest_int('min_samples_leaf', 1, 50),
        'max_features'     : trial.suggest_float('max_features', 0.2, 1.0),
        'ccp_alpha'        : trial.suggest_float('ccp_alpha', 0.0, 0.5),
        'n_jobs'           : -1,
        'random_state'     : SEED
    }
    modelo = RandomForestRegressor(**params)
    scores = cross_val_score(
        estimator = modelo, X=X_train, y=y_train,
        cv        = RepeatedKFold(n_splits=5, n_repeats=3, random_state=123),
        scoring   = 'neg_root_mean_squared_error',
        n_jobs    = -1
    )
    return np.mean(scores)

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=30, show_progress_bar=True, timeout=60*5)

print('\n Mejores hiperparámetros (Optuna):', study.best_params)
print(f'   Mejor score (neg RMSE): {study.best_value:.4f}')
print(f'   Mejor RMSE estimado  : {-study.best_value:.4f} puntos de estrés')
```

```
                        0.0/419.5 kB
? eta -:--:--
                      409.6/419.5
kB 14.4 MB/s eta 0:00:01
                      419.5/419.5 kB
9.1 MB/s eta 0:00:00
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
  0%|          | 0/30 [00:00<?, ?it/s]
```

```
 Mejores hiperparámetros (Optuna): {'n_estimators': 100, 'max_depth': 3,
'min_samples_split': 32, 'min_samples_leaf': 1, 'max_features':
0.7600081312575583, 'ccp_alpha': 0.0001536677001042347}
   Mejor score (neg RMSE): -2.8955
   Mejor RMSE estimado  : 2.8955 puntos de estrés
```